J E L L S T R I P

Self-Hosted Home Infrastructure

Combined Technical Design & Operations Document

Version 1.0

Submitted by:

Devang Chaturvedi — Reg. No. 23BCT0151

B.Tech Computer Science Engineering

VIT Vellore

Host Node: nas | Primary Host: 192.168.1.90 | Management: 192.168.1.200:8006

Status: OPERATIONAL (80% Stack Active, Daily Use)

Abstract

Jellstrip is a fully self-hosted home infrastructure project designed, built, and operated by Devang Chaturvedi on a budget desktop PC converted into a production-grade home server. The system is built upon Proxmox VE as the bare-metal hypervisor, running a Docker-based Linux Container (LXC) that hosts a unified stack of self-hosted services including music streaming, photo management, video streaming, network-wide ad blocking, private cloud storage, and a Soulseek music automation pipeline controlled via Telegram.

The infrastructure is structured around five core subsystems: the Hypervisor and Host Layer (Proxmox VE on NVMe SSD), the Network and DNS Layer (AdGuard Home, Unbound recursive resolver, Tailscale mesh VPN), the Media and Entertainment Stack (Jellyfin, dual Navidrome instances, Nicotine+ Soulseek client), the Storage and Cloud Layer (Samba SMB private cloud, Immich photo management), and the YouTube Alternative Stack (Piped, Materialious, Invidious with Companion). The system is entirely headless and managed remotely via SSH and browser-based interfaces, accessible both on the local network and globally over Tailscale.

This document provides a comprehensive technical record of the design decisions, deployment procedures, incident reports, engineering challenges, workarounds, and current operational status of every subsystem in the Jellstrip infrastructure.

System Architecture Overview

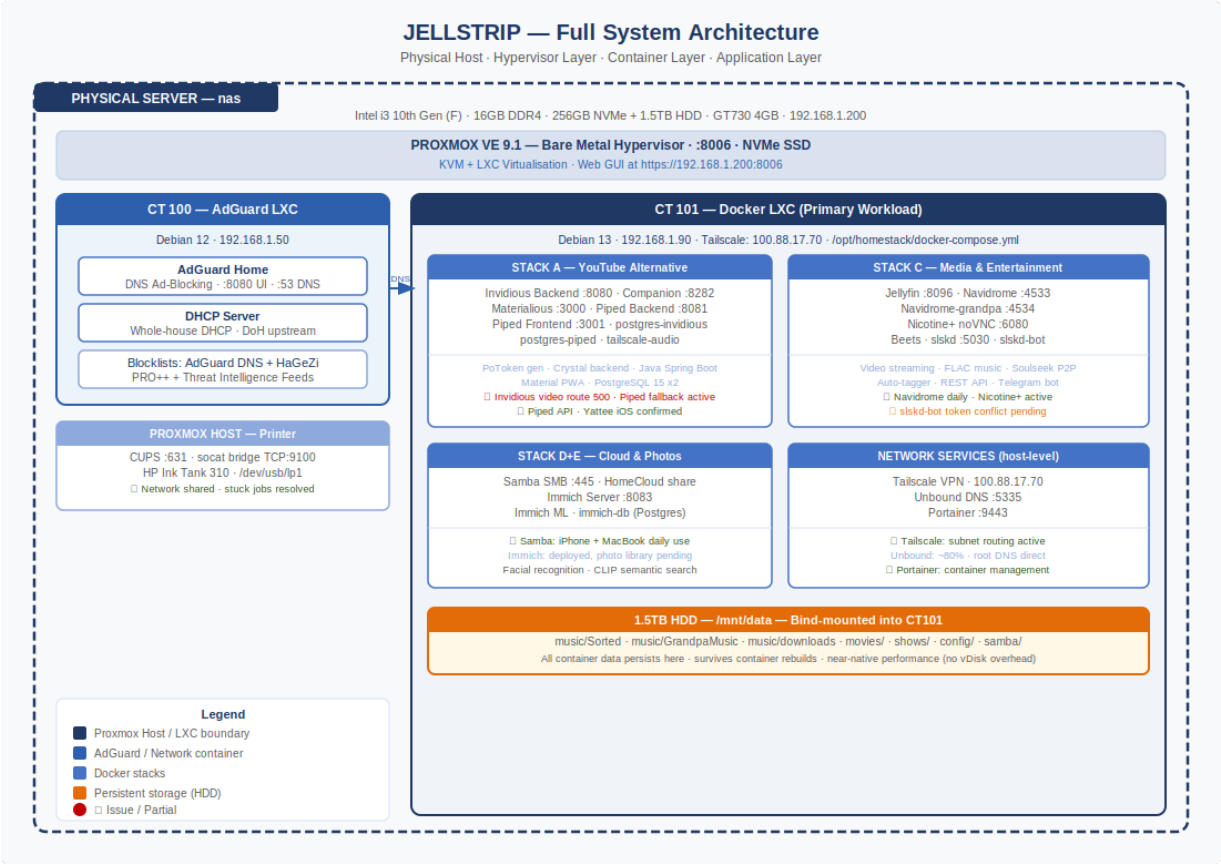

Figure 1: System Architecture Overview

Hardware Specification

Component	Specification
CPU	Intel Core i3 10th Gen (F-Series) — No Integrated GPU
Motherboard	H410 Chipset
RAM	16 GB DDR4 (Single DIMM, Slot DIMM_A2)
GPU	NVIDIA GeForce GT 730 4GB (Kepler Architecture)
Primary Storage	256 GB NVMe SSD — Proxmox OS + Container Root
Secondary Storage	1.5 TB HDD — Media, Music, Photos, Data (mounted at /mnt/data)
Connectivity	Gigabit Ethernet (primary) + WiFi/BT USB Dongle
Network IP	192.168.1.200 (Proxmox Host) | 192.168.1.90 (Docker LXC)
Tailscale IP	100.88.17.70 (permanent mesh address)

Container Architecture

The system operates on a two-tier containerisation model. The Proxmox hypervisor runs directly on bare metal on the NVMe SSD. Two primary Linux Containers (LXCs) are provisioned within Proxmox:

- CT 100 — AdGuard Home LXC (Debian 12, dedicated DNS/ad-blocking container, IP: 192.168.1.50)
- CT 101 — Docker LXC (Debian 13, primary workload container, IP: 192.168.1.90, all application services deployed here via Docker Compose)

All persistent data for CT 101 services is stored on the 1.5TB HDD, bind-mounted into the container from the Proxmox host. The HDD is mounted at /mnt/data on the host and passed directly into the Docker LXC, providing near-native storage performance without the overhead of virtual disk files.

Service Map

Service	Port	Status	Function
Proxmox VE	8006	✓ Active	Hypervisor / host management
AdGuard Home	8080	✓ Active	Network-wide DNS ad blocking
Unbound DNS	5335	✓ Active	Recursive DNS resolver (no third-party upstream)
Tailscale	—	✓ Active	Mesh VPN, remote access, CGNAT bypass
Portainer	9443	✓ Active	Docker container management UI
Jellyfin	8096	✓ Deployed	Media streaming server (movies, shows)
Navidrome (Personal)	4533	✓ Active	Music streaming — ~100 FLAC files
Navidrome (Grandpa)	4534	✓ Active	Music streaming — ~1000 songs (family)
Nicotine+	6080	✓ Active	Soulseek P2P music downloader (noVNC GUI)
Beets	—	✓ Active	Automatic music library tagger
slskd	5030	✓ Connected	Headless Soulseek engine (REST API)
slskd-bot	—	⚠ Bot conflict	Telegram bot for Soulseek automation
Samba (SMB)	445	✓ Active	Private cloud storage (iPhone + MacBook)
Immich	8083	✓ Deployed	Photo management (Apple Photos alternative)
Piped Backend	8081	✓ Running	YouTube API backend
Piped Frontend	3001	✓ Running	YouTube PWA frontend
Materialious	3000	⚠ 500 errors	Material Design YouTube frontend
Invidious Backend	8080	⚠ Video 500s	YouTube data extraction backend

CUPS (Printer)	631	✓ Active	HP Ink Tank 310 network sharing

Section 1 — Hardware Bring-Up and Proxmox Installation

1.1 Overview

Section 1 documents the complete physical assembly, initial hardware validation, data recovery operations, and Proxmox VE installation process. This section covers all challenges encountered during the hardware bring-up phase and the workarounds applied to achieve a stable hypervisor installation.

1.2 Hardware Assembly and Initial Validation

The server hardware was assembled from components sourced independently. Upon initial power-on, the primary GPU (NVIDIA GT 730) exhibited no video output via its HDMI port despite the system correctly powering on and completing POST, as confirmed by audible POST beeps. The issue was traced to the GPU's HDMI output being non-functional on this specific unit. Validation was achieved by switching to the motherboard's VGA output, which produced a stable display signal, confirming the system was booting correctly.

Root Cause: GT 730 GPU HDMI output non-functional on this unit.

Workaround: VGA output used for initial setup. Server subsequently configured as fully headless — no monitor required for ongoing operation.

1.3 Legacy SSD Data Recovery

Before proceeding with any destructive storage operations, an old SSD containing important family data (videos, personal documents) was connected internally to the system. The Windows installation on the SSD was password-protected, preventing direct access through Windows boot.

1.3.1 Recovery Environment Selection

Ubuntu Live was initially attempted as a recovery environment but encountered graphical freezes during boot, attributed to NVIDIA GT 730 driver initialisation conflicts with the Live USB environment. Linux Mint XFCE was selected as an alternative, as its XFCE desktop environment has a significantly lower GPU dependency and booted successfully without graphical issues.

1.3.2 Recovery Workflow

1. Booted Linux Mint XFCE from USB drive
2. Opened file manager and navigated to the Windows partition (automatically mounted as read-only)
3. Identified Windows user profile directories: Videos, Desktop, Downloads, Documents
4. Copied all identified family data to the 1.5TB HDD as a staging backup
5. Verified successful file access and integrity before proceeding

Outcome: 100% of target family data successfully recovered prior to any storage modifications.

1.4 Proxmox VE Installation

1.4.1 Installation Preparation

Proxmox VE 9.1 ISO was downloaded from the official Proxmox repository and flashed to a USB drive using BalenaEtcher. The 256GB NVMe SSD was selected as the installation target.

1.4.2 Installer Freeze — GPU Driver Conflict

The Proxmox graphical installer froze during driver initialisation on the first boot attempt. This is a known issue when the NVIDIA GT 730 (Kepler architecture) is present and the kernel attempts to load the nouveau open-source driver during the installer's graphical phase.

Resolution: The kernel boot parameters were edited at the GRUB boot menu by pressing 'e' on the Proxmox installer entry and appending the following parameter:

```
nomodeset
```

This parameter disables kernel mode-setting, preventing the nouveau driver from attempting to initialise the GPU framebuffer during installation. The installer completed successfully after this modification.

1.4.3 Installation Configuration

Parameter	Value
Install Target	256 GB NVMe SSD
Filesystem	EXT4 (ZFS/RAID avoided for initial simplicity)
Hostname	nas.local
Initial Management IP	192.168.100.2 (later corrected — see Section 2)
Gateway	192.168.100.1
DNS	1.1.1.1 (Cloudflare, later superseded by AdGuard)

1.4.4 Boot Order Conflict

After installation completed, the system booted into an old Ubuntu installation residing on the 1.5TB HDD rather than the newly installed Proxmox on the NVMe SSD. This occurred because the HDD's GRUB bootloader had higher priority in the BIOS boot order than the NVMe SSD.

Resolution: The BIOS boot order was corrected to prioritise the NVMe SSD. The HDD boot sector was subsequently overwritten when the HDD was reformatted for media storage use. Proxmox boot confirmed successful at https://192.168.100.2:8006/

1.5 Engineering Assessment — Hardware and Installation

Category	Detail

Strengths	i3 10th Gen provides adequate compute for 15+ concurrent Docker containers. NVMe SSD ensures fast OS/container operations. 1.5TB HDD provides sufficient media storage for current use case.
Limitations	F-Series CPU has no iGPU — no hardware video transcoding (Intel QuickSync unavailable). GT 730 Kepler architecture lacks NVENC for HEVC. Single HDD with no redundancy — no RAID protection. Single RAM stick — no dual-channel performance benefit.
Pending	Gigabit switch procurement to eliminate 100Mbps Fast Ethernet bottleneck. Additional HDD for storage redundancy.

Section 2 — Critical Infrastructure Incident: Post-Mortem Report

2.1 Executive Summary

Following the initial Proxmox installation, the system experienced a compounding sequence of critical failures resulting in complete display blackout, motherboard POST failure, and total network isolation. The incident was caused by four independently interlocking root causes that triggered each other in sequence. Full recovery was achieved through systematic physical and software-level intervention. The system emerged from this incident in a significantly improved state, fully updated and properly configured.

Field	Detail
Host Node	nas
Date / Time	May 19, 2026
Target OS	Proxmox VE (Debian Trixie Base)
Current Status	RESOLVED / STABLE

2.2 Root Cause Analysis

2.2.1 Root Cause A — Unpatched Operating System (Software Trigger)

The system had 177 uninstalled operating system updates queued at the time of the incident. This backlog contained historical bugs within core networking libraries, specifically libnftables1 and nftables. When background network packets circulated through the kernel's bridge layer, the kernel intermittently hit memory address space leaks, generating structural Segmentation Faults within kernel space.

2.2.2 Root Cause B — GPU Framebuffer Collision (Display Blackout Cause)

Because the Intel i3 F-Series processor has no integrated graphics, the Linux kernel maps its terminal console exclusively through the dedicated NVIDIA GT 730 framebuffer. When a command such as 'clear' was issued, the operating system attempted an instantaneous wholesale rewrite of the active GPU memory blocks. Simultaneously catching a network segmentation fault mid-rewrite caused the graphics interface engine to fail completely, dropping its active monitor signal and producing a blank screen.

2.2.3 Root Cause C — NVRAM/BIOS Corruption (Motherboard Lock)

The violent mid-operation GPU/system crash caused residual electrical and data instability that corrupted the motherboard's volatile NVRAM configuration profile. Upon subsequent forced hardware restarts, the MSI motherboard's safety routines flagged this state as a critical boot blocker. The DRAM EZ Debug LED on the MSI board illuminated solid white, and the system halted before power could reach the monitor or GPU channels, creating a pre-POST lock.

2.2.4 Root Cause D — Subnet Isolation Standoff (Network Unreachability)

The Proxmox host's static network configuration was locked onto an alien subnet pool (192.168.100.2), while the local home router infrastructure was broadcasting on the 192.168.1.x layout. Simultaneously, the

managing MacBook's network address had defaulted to an unusable configuration (0.0.0.0 on a 192.168.100.1 routing boundary). The devices were entirely unable to communicate across the local LAN.

2.3 Resolution Workflow

Phase 1 — Breaking the Physical Board Lockout

6. Capacitive Drain: Disconnected AC power from the wall and held the physical power button for 15 seconds to flush residual voltage from board capacitors.
7. RAM Reset and Cleaning: Extracted the single DDR4 memory stick. Cleaned the gold contacts to eliminate microscale oxidation and static variance. Repositioned the stick strictly into the primary slot (Slot 2 / DIMM_A2) to guarantee correct memory timing initialisation.
8. CMOS Clearance: Applied a conducting screwdriver tip across the JBAT1 header pins for 15 seconds. This wiped the scrambled NVRAM configuration table, cleared the white DRAM EZ Debug LED, and restored the primary hardware video signal path.

Phase 2 — Local Subnet Alignment

Once console access was regained, the physical interface network profile was corrected:

```
nano /etc/network/interfaces
auto vmbr0
iface vmbr0 inet static
    address 192.168.1.200/24
    gateway 192.168.1.1
    bridge-ports nic0
    bridge-stp off
    bridge-fd 0
systemctl restart networking
```

Phase 3 — Client Workspace Recovery

The managing MacBook's manual IP selection was moved from 0.0.0.0 back to Automatic DHCP, restoring standard SSH and web UI access paths. MacBook received IP 192.168.1.83 from the router.

Phase 4 — Repository Cleaning and Full System Update

Broken apt enterprise repository sources that were blocking updates were removed:

```
rm -f /etc/apt/sources.list.d/ceph.sources
rm -f /etc/apt/sources.list.d/pve-enterprise.sources
apt-get clean && apt-get update && apt-get dist-upgrade -y
```

All 177 pending system updates were successfully applied. The system was rebooted into the stable, fully patched kernel.

2.4 Final System State Matrix

Subsystem	Incident State	Resolved State
Motherboard POST	Halted — Solid White DRAM EZ Debug LED	Clean Initialisation — All LEDs Clear
Network Subnet	Isolated on 192.168.100.2 block	Active on 192.168.1.200/24
Repository Config	Malformed enterprise files, broken characters	Clean no-subscription sources only

OS Packages	177 unstable packages out of date	Fully upgraded stable dist-kernel release
Management Access	Local screen/keyboard + pin short circuits	100% headless — SSH and Web GUI at 192.168.1.200:8006

Section 3 — Network and DNS Layer

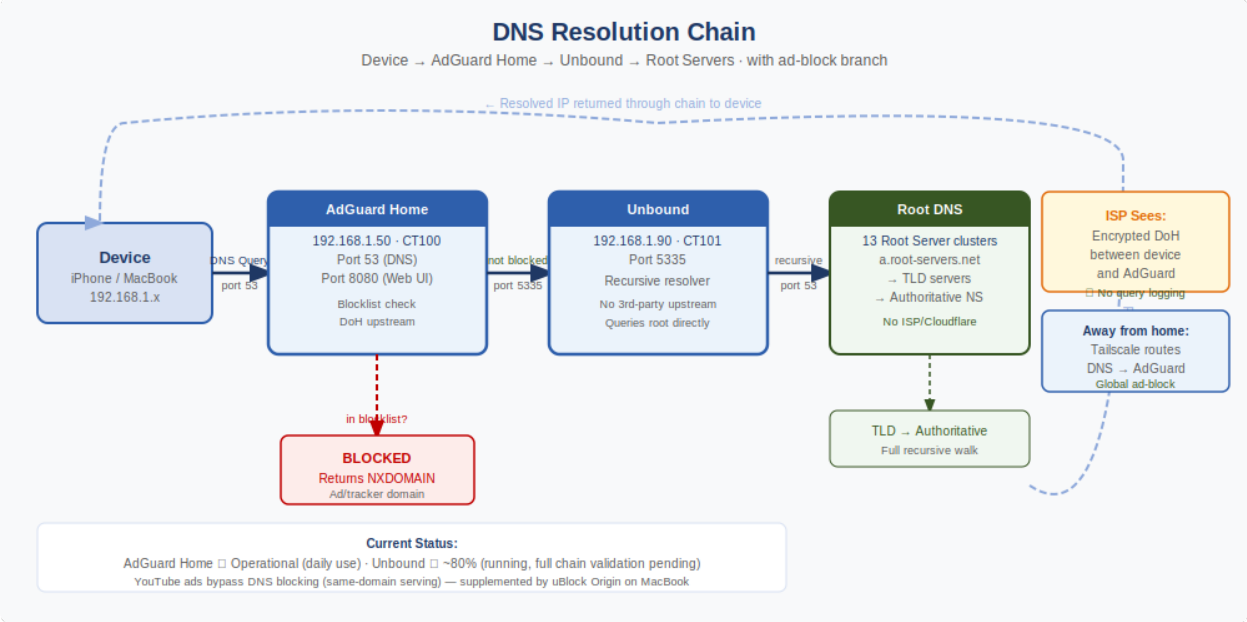

Figure 2: DNS Resolution Chain

3.1 Overview

The network layer is the foundational access control and privacy layer of the Jellstrip infrastructure. It comprises three tightly integrated components: the Nokia G-2425G-A ONT/router provided by Airtel (with locked administrator credentials), AdGuard Home as the network-wide DNS ad-blocking and DHCP server, and Unbound as the recursive DNS resolver providing zero-trust upstream DNS resolution. Tailscale provides remote access via encrypted mesh VPN.

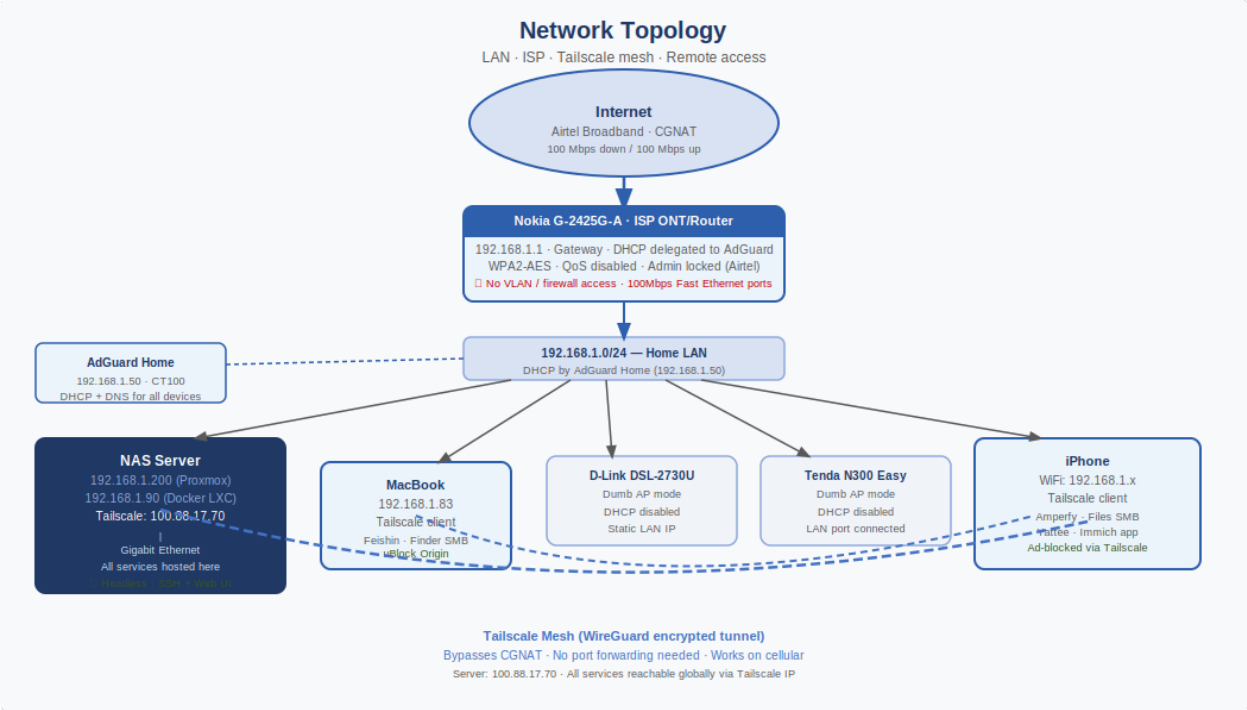

Figure 3: Network Topology

3.2 ISP Router — Nokia G-2425G-A

The Nokia G-2425G-A is an Airtel-provided ONT/router combination unit. Airtel provides only user-level credentials to subscribers, with the administrator account locked at the ISP level. This prevents access to advanced routing features including VLAN configuration, firewall rules, and full DHCP control.

3.2.1 Configuration Performed (User-Level Access)

- WiFi Security Mode: Changed from WPA/WPA2 Mixed Mode to pure WPA2-AES. Mixed mode maintained a WPA1 fallback that created security vulnerabilities and reduced throughput.
- QoS: Disabled. The Nokia ONT's processor is insufficiently powerful to perform per-packet Quality of Service inspection without introducing measurable latency. Disabling QoS improved raw throughput.
- LAN DNS: Pointed to AdGuard Home container IP (192.168.1.50) to route all household DNS queries through the ad-blocking layer.

3.2.2 Access Point Configuration

Two secondary routers (D-Link DSL-2730U and Tenda N300 Easy) were reconfigured as pure Access Points by disabling their DHCP servers, setting static LAN IPs outside the router's DHCP pool, and connecting them via LAN port (not WAN port). This eliminated double NAT and unified all devices onto the 192.168.1.x subnet.

3.3 AdGuard Home

3.3.1 Deployment

AdGuard Home was deployed inside Proxmox LXC Container 100 (CT 100), running Debian 12. The installation was performed via the official AdGuard Home install script:

```
curl -s -L
https://raw.githubusercontent.com/AdguardTeam/AdGuardHome/master/scripts/install.sh |
sh
```

3.3.2 Installation Issue — Port Conflict

The install script immediately starts AdGuard Home as a background daemon upon completion. When the browser-based setup wizard was opened, it attempted to bind to the same ports (3000 and 53) that the background daemon had already claimed, producing a 'bind: address already in use' error and a network error in the wizard.

Resolution: The background service was stopped to free the ports, allowing the wizard to complete configuration and write its configuration file:

```
/opt/AdGuardHome/AdGuardHome -s stop
# Complete wizard configuration
/opt/AdGuardHome/AdGuardHome -s start
```

3.3.3 Configuration

Setting	Value
Web Interface Port	8080
DNS Port	53 (standard)
Upstream DNS	https://dns.cloudflare.com/dns-query (DoH) → Unbound (port 5335) as primary
Bootstrap DNS	1.1.1.1, 8.8.8.8
DHCP Server	Enabled — AdGuard acts as DHCP server for the house
Blocklist — Primary	AdGuard DNS Filter
Blocklist — Extended	HaGeZi Multi PRO++
Blocklist — Security	Threat Intelligence Feeds (TIF)
DNS-over-HTTPS	Enabled — ISP cannot observe domain queries

3.3.4 YouTube Ad Limitation

AdGuard Home, as a DNS-level blocker, is unable to block YouTube video advertisements. YouTube serves both video content and advertisements from the same domain (youtube.com and googlevideo.com). Blocking these domains at the DNS level would disable YouTube entirely. This is a fundamental architectural limitation of DNS-level ad blocking. The workaround implemented was to use uBlock Origin browser extension on the MacBook for YouTube-specific ad suppression, while AdGuard continues to protect all other traffic network-wide.

3.4 Unbound Recursive DNS Resolver

Unbound is deployed as a Docker container within the homestack on port 5335. It serves as the upstream resolver for AdGuard Home, replacing reliance on any third-party DNS provider. Unbound queries the global DNS root servers directly, ensuring that no external entity (Cloudflare, Google, ISP) ever receives a log of the household's DNS queries.

3.4.1 DNS Chain Architecture

The complete DNS resolution chain is:

- Device sends DNS query to AdGuard Home (port 53 at 192.168.1.50)
- AdGuard Home checks query against blocklists — blocks if matched, returns NXDOMAIN
- If not blocked, AdGuard Home forwards to Unbound (port 5335 on 192.168.1.90)
- Unbound performs full recursive resolution by querying DNS root servers directly
- Response returned through the chain to the requesting device

Current Operational Status: ~80% functional. Unbound is deployed and responding. Full integration validation pending.

3.5 Tailscale Mesh VPN

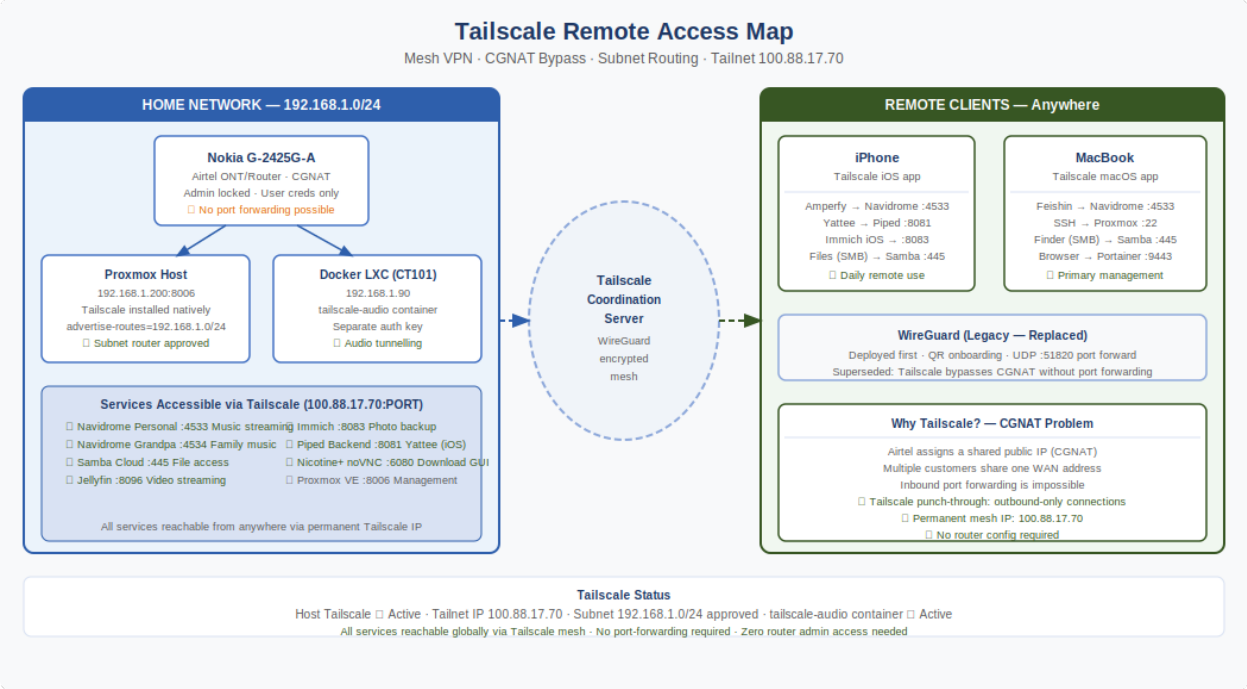

Figure 7: Tailscale Remote Access Map

3.5.1 Deployment

Tailscale was installed directly on the Docker LXC host (CT 101 at 192.168.1.90) using the official installation script:

```
curl -fsSL https://tailscale.com/install.sh | sh
sudo tailscale up --advertise-routes=192.168.1.0/24
```

3.5.2 Configuration

- Subnet routing (192.168.1.0/24) approved in Tailscale Admin Console
- 'Use Tailscale Subnets' enabled on MacBook client
- Permanent server Tailscale IP: 100.88.17.70
- Tailscale installed on iPhone and MacBook
- AdGuard Home Tailscale IP set as global DNS nameserver in Tailscale Admin Console — all devices receive ad-blocking protection even when away from home network

3.5.3 WireGuard (Superseded)

WireGuard was the initial VPN solution deployed before Tailscale. A WireGuard container was deployed, QR codes generated for phone onboarding, and port forwarding was configured on the Nokia router (UDP 51820). WireGuard was superseded by Tailscale because Tailscale bypasses CGNAT automatically without requiring port forwarding on the ISP router, providing a simpler and more reliable remote access solution.

3.5.4 Tailscale Container (tailscale-audio)

An additional Tailscale container (tailscale-audio) is deployed within the Docker Compose stack specifically for audio service tunnelling, allowing Navidrome to be accessed externally with a dedicated Tailscale auth key separate from the host-level Tailscale installation.

3.6 Engineering Assessment — Network Layer

Category	Detail
Strengths	Full DNS chain from AdGuard → Unbound → Root provides enterprise-grade privacy. Tailscale CGNAT bypass requires zero router port forwarding. DoH prevents ISP from observing domain queries. AdGuard-as-DHCP ensures all household devices use the filtering stack.
Limitations	Nokia router admin lock prevents VLAN configuration and advanced firewall rules. 100Mbps Fast Ethernet bottleneck on some network segments. YouTube ads bypass DNS-level blocking due to same-domain ad serving.
Pending	Gigabit switch to eliminate Fast Ethernet bottleneck. OPNsense VM on Proxmox for full routing control (planned, not deployed).

Section 4 — Media and Entertainment Stack

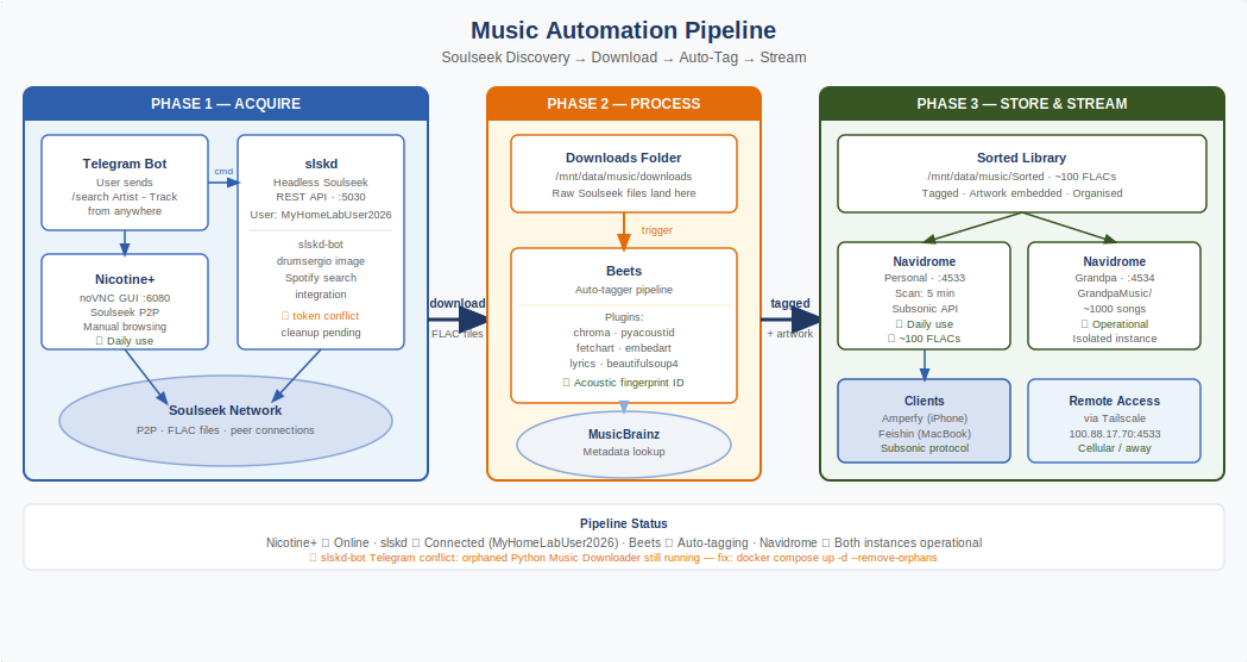

Figure 4: Music Automation Pipeline

4.1 Overview

The media and entertainment stack comprises five interconnected services: Jellyfin for video media streaming, two independent Navidrome instances for music streaming, Nicotine+ as the primary Soulseek P2P music acquisition interface, Beets for automatic music library organisation and tagging, and the slskd engine with Telegram bot for automated music search and download. This stack represents the most actively used portion of the Jellstrip infrastructure.

4.2 Jellyfin Media Server

4.2.1 Deployment Configuration

Jellyfin is deployed as a Docker container within the homestack on port 8096. Media is sourced from the 1.5TB HDD via bind mounts:

```
jellyfin:
  image: jellyfin/jellyfin:latest
  container_name: jellyfin
  ports:
    - "8096:8096"
  volumes:
    - /mnt/data/config/jellyfin:/config
    - /mnt/data/movies:/movies
    - /mnt/data/shows:/shows
```

4.2.2 Transcoding Constraints

The i3 F-Series processor has no Intel QuickSync capability, precluding hardware-accelerated video transcoding via iGPU. The GT 730 GPU (Kepler architecture) lacks NVENC support for HEVC encoding. All transcoding must be performed via software on the CPU's 4 cores. This is acceptable for single-stream 1080p playback but may become a constraint for simultaneous multi-stream or 4K direct transcoding scenarios. Direct Play is the preferred playback method to eliminate transcoding overhead entirely.

Remote Access: Jellyfin is accessible remotely at http://100.88.17.70:8096 via Tailscale.

Current Status: Deployed and operational. Media library population is in progress.

4.3 Navidrome Music Streaming

4.3.1 Architecture Decision — Dual Instance

Two independent Navidrome instances are deployed within the same Docker Compose stack, each on a separate port with separate configuration and music library directories. This architecture was chosen to provide dedicated, isolated music libraries for two separate users without requiring user account management within a single shared instance.

4.3.2 Personal Instance (Port 4533)

The personal Navidrome instance serves the primary user's music collection of approximately 100 FLAC audio files downloaded via Nicotine+ from the Soulseek network. The music library is stored at /mnt/data/music/Sorted and is scanned every 5 minutes.

Parameter	Value
Container Name	navidrome
Port	4533
Music Path	/mnt/data/music/Sorted
Config Path	/mnt/data/config/navidrome
Scan Schedule	Every 5 minutes
iOS Client	Amperfy
Desktop Client	Feishin (MacBook)
Content	~100 FLAC files (lossless audio)
Status	Fully operational — daily use

4.3.3 Grandfather's Instance (Port 4534)

A dedicated Navidrome instance was deployed specifically for the primary user's grandfather's music collection, comprising approximately 1,000 songs. This instance is entirely isolated from the personal instance, with separate configuration directories and a dedicated music library path.

Parameter	Value
Container Name	navidrome-grandpa
Port	4534
Music Path	/mnt/data/music/GrandpaMusic
Config Path	/mnt/data/config/navidrome-grandpa
Content	~1,000 songs
Status	Fully operational

4.4 Nicotine+ Soulseek Client

4.4.1 Overview

Nicotine+ is the primary music acquisition interface in the Jellstrip infrastructure. It is deployed as a Docker container using the binhex/arch-nicotineplus image, which packages the full Nicotine+ desktop application with a noVNC web server, providing browser-based GUI access without requiring a physical display or X11 forwarding.

4.4.2 Deployment Issue — Disk Space Exhaustion

The binhex/arch-nicotineplus image is approximately 1.5GB in size, as it includes a full desktop environment (Arch Linux base with a virtual framebuffer and VNC server). During the initial pull attempt, the Docker LXC container's system disk (24GB partition on the NVMe SSD) became exhausted before the image could be fully extracted and unpacked. The container crashed during startup.

Resolution: The 1.5TB HDD was correctly mounted and bind-mounted into the Docker LXC. After confirming /mnt/data was available with sufficient space, the image pull was retried and succeeded.

4.4.3 Access and Operation

Nicotine+ is accessed via browser at http://192.168.1.90:6080/vnc.html, which loads the full noVNC desktop interface. The Soulseek network status displays as Online. Downloads are directed to /mnt/data/music/downloads, from where Beets processes and organises the files into /mnt/data/music/Sorted.

4.4.4 UPnP Warning

The Nicotine+ container logs a persistent UPnP warning: 'Failed to forward external port 2234: No UPnP devices found'. This occurs because the Nokia G-2425G-A does not respond to UPnP port mapping requests. Port 2234 is Nicotine+'s preferred external listening port for incoming Soulseek peer connections. The absence of UPnP port forwarding limits the ability to accept incoming connections from other Soulseek peers but does not prevent outbound search and download functionality. This warning does not affect operational status.

Current Status: Online and actively used for FLAC music downloads.

4.5 Beets Music Library Manager

Beets is deployed as a Docker container providing automatic music library organisation and metadata tagging. It operates on the music download directory and applies comprehensive metadata enrichment before files are moved to the sorted library directory.

Parameter	Value
Image	lscr.io/linuxserver/beets:latest
Plugins	lyrics, chroma, fetchart, embedart, pyacoustid, beautifulsoup4
Input Path	/mnt/data/music/downloads
Output Path	/mnt/data/music/Sorted
slskd Integration	Auto-import triggered on slskd download completion via SLSKD_INTEGRATIONS_SCRIPTS_TRIGGER_BEETS_RUN_ARGS

4.6 slskd + Telegram Bot Music Automation

4.6.1 Evolution of the Music Automation Pipeline

The music automation pipeline underwent three distinct evolutionary phases before arriving at its current architecture:

Phase 1 — Custom Python Script (Music Downloader v0.10.5): A custom Python-based music downloader was initially implemented. It used file-polling to monitor download directories for changes and triggered Telegram notifications. This approach was fragile, prone to crashes, and created race conditions when multiple downloads were in progress simultaneously.

Phase 2 — Nicotine+ Container with Sidecar Bot: An attempt was made to run the existing nicotine-plus GUI container alongside a Telegram bot that would interact with it programmatically. The binhex/arch-nicotineplus container wraps a full desktop application and is not designed for programmatic API access. Modifying its entrypoints to inject commands caused the VNC server to fail, rendering the GUI inaccessible.

Phase 3 — slskd API Engine + drumsergio Bot (Current): The architecture was redesigned around slskd, a purpose-built headless Soulseek client with a full REST API. The drumsergio/telegram-slskd-local-bot container interfaces with the slskd API directly, providing stable Telegram-based search and download commands without requiring any interaction with GUI containers.

4.6.2 slskd Configuration

```
slskd:
  image: slskd/slskd:latest
  ports:
    - "5030:5030"
  environment:
    - SLSKD_HTTP_AUTH_METHOD=username-password
```

```
    - SLSKD_HTTP_USERNAME=admin
    - SLSKD_SLSK_USERNAME=MyHomeLabUser2026
    - SLSKD_REMOTE_CONFIGURATION=true
```

4.6.3 Issues and Resolutions

Issue 1 — Soulseek Authentication Failure: [WRN] Not connecting to the Soulseek server; username and/or password invalid. The SLSKD_SLSK_USERNAME and SLSKD_SLSK_PASSWORD environment variables were missing from the initial deployment. Added and the engine successfully authenticated as MyHomeLabUser2026.

Issue 2 — 401 Unauthorized on Web UI: The slskd Web UI returned HTTP 401. Resolution: SLSKD_HTTP_USERNAME and SLSKD_HTTP_PASSWORD environment variables were added to the slskd service block.

Issue 3 — telegram.error.Conflict: The Telegram bot reported a Conflict error indicating two processes were simultaneously polling the same Telegram bot token. The orphaned custom Python 'Music Downloader v0.10.5' script was still running as a container alongside the new slskd-bot, both competing for the same Telegram token. Resolution: Remove the legacy nicotine-tg-bot service from docker-compose.yml and run docker compose up -d --remove-orphans.

Current Status: slskd engine connected and authenticated. Telegram bot conflict resolution pending final cleanup.

Section 5 — Storage and Private Cloud Layer

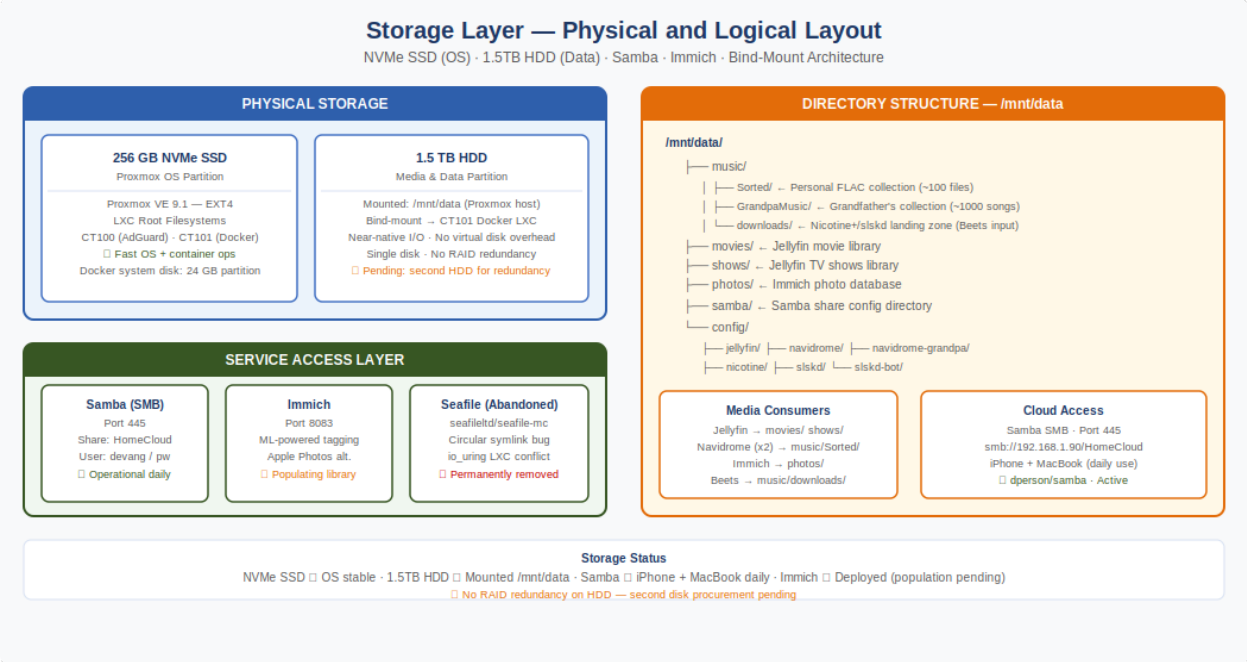

Figure 5: Storage Layer

5.1 Overview

The storage and private cloud layer comprises two services: Samba for SMB-based private cloud file access (the primary production storage solution) and Immich for AI-powered photo management. This section also documents the failed Seafile deployment that preceded the Samba implementation, including the full incident report and the engineering decision to abandon Seafile in favour of the simpler Samba architecture.

5.2 Seafile Deployment — Incident and Abandonment

5.2.1 Deployment Attempt

Seafile was initially selected as the private cloud solution on the basis of its performance advantages over Nextcloud (written in C/Python rather than PHP, Git-like block-based file storage, superior sync client). The deployment used the seafileltd/seafile-mc image paired with a MariaDB database container.

5.2.2 Critical Error — Circular Symlink Loop

Upon container startup, Seafile's Nginx web server encountered a fatal circular symlink error:

```
nginx: [alert] could not open error log file: open() "/var/log/nginx/error.log"
failed (40: Too many levels of symbolic links)
open() "/var/log/nginx/seahub.access.log" failed (40: Too many levels of symbolic
links)
```

The error recurred on every container restart, repeating indefinitely. The seafile-mc image's initialisation script creates internal symlinks for its log directory structure. When the shared volume was mounted from

the host, Docker's volume binding conflicted with the container's internal symlink creation, producing a recursive symlink chain that Nginx could not traverse.

5.2.3 Resolution Attempts

- Fresh volume paths: Multiple attempts using clean host directories (/opt/homestack/seafile-data, /opt/homestack/seafile-fresh, /opt/homestack/seafile-storage) — all reproduced the same error.
- Volume prune: docker volume prune -f was run to eliminate cached volume state — error persisted.
- Hardcoded absolute paths: Replaced all relative volume paths with absolute paths — no change.
- Removal of healthcheck: seafile-db healthcheck removed to prevent dependency timeout — Nginx error remained.

5.2.4 Engineering Decision — Abandon Seafile

After multiple failed fresh deployments, the engineering decision was made to abandon the Seafile implementation entirely. The seafile-mc Docker image has a known incompatibility with certain LXC-based Docker environments due to the io_uring restriction (kernel.io_uring_disabled = 2) present in the Proxmox LXC container configuration, which also affected MariaDB initialisation. Given the time cost of continued debugging and the availability of a simpler alternative (Samba), Seafile was permanently removed.

```
docker compose down seafile seafile-db
rm -rf /opt/homestack/seafile-data /opt/homestack/seafile-fresh
/opt/homestack/seafile-storage
docker volume prune -f
```

5.3 Samba SMB Private Cloud

5.3.1 Architecture Decision

Samba was selected as the replacement private cloud solution. While Samba lacks the web interface and sync client sophistication of Seafile or Nextcloud, it provides a universally compatible SMB/CIFS file share that is natively supported by iOS Files app, macOS Finder, and Windows Explorer without requiring any third-party client installation. The entire 1.5TB HDD (/mnt/data) is exposed as a single shared volume, providing unrestricted access to all media, music, photos, and project files.

5.3.2 Deployment Configuration

```
samba:
  image: dperson/samba:latest
  container_name: samba
  network_mode: host
  environment:
    USER: homestack;SecureCloudPass123!
    SHARE: HomeCloud;/data;yes;no;no;homestack
    GLOBAL: "min protocol = SMB2\ndfree command = /usr/bin/df /data"
  volumes:
    - /mnt/data/samba:/etc/samba
    - /mnt/data:/data
```

5.3.3 Configuration Issues

Issue — Permissions: Initial deployment failed because the /mnt/data directories were owned by root, and the homestack user did not have write permissions. Files could be browsed but not created or modified.

Resolution: Directory ownership was corrected with chown -R homestack:homestack /mnt/data, and permissions were set to 755 to allow write access for the homestack user.

5.3.4 Client Access

Device	Access Method	Status
MacBook	Finder → Go → Connect to Server → smb://192.168.1.90	Daily use
iPhone	Files app → Browse → Connect to Server → smb://192.168.1.90	Daily use

Current Status: Fully operational. Used daily as primary private cloud storage from both iPhone and MacBook.

5.4 Immich Photo Management

5.4.1 Selection Rationale

Immich was selected over Nextcloud and other alternatives as the photo management solution because it is explicitly designed to replicate the experience of Apple Photos or Google Photos in a self-hosted environment. Key features include: a polished native iOS and Android app with automatic background photo upload, a scrubbable timeline interface, Live Photo support, HDR video playback, location metadata mapping, and local on-device AI processing for facial recognition and semantic search (allowing text-based photo search such as 'circuit board' or 'sunset').

5.4.2 Stack Architecture

The Immich deployment comprises three containers working in concert:

- immich-server: Core application server (port 8083) — handles API requests, photo ingestion, and client communication
- immich-machine-learning: Local ML inference container — performs facial recognition, object detection, and CLIP embeddings for semantic search entirely on-device
- immich-db: Dedicated PostgreSQL 15 instance — stores photo metadata, face clusters, album data, and search indices

5.4.3 Deployment Configuration

```
immich-server:
  image: ghcr.io/immich-app/immich-server:release
  ports:
    - "8083:3001"
```

```
  environment:
    - DB_HOSTNAME=immich-db
    - DB_USERNAME=immich
    - DB_DATABASE_NAME=immich
    - IMMICH_MACHINE_LEARNING_URL=http://immich-machine-learning:3001
```

Remote Access: Immich is accessible at http://100.88.17.70:8083 via Tailscale.

Current Status: Deployed and operational. Photo library population in progress.

Section 6 — YouTube Alternative Stack

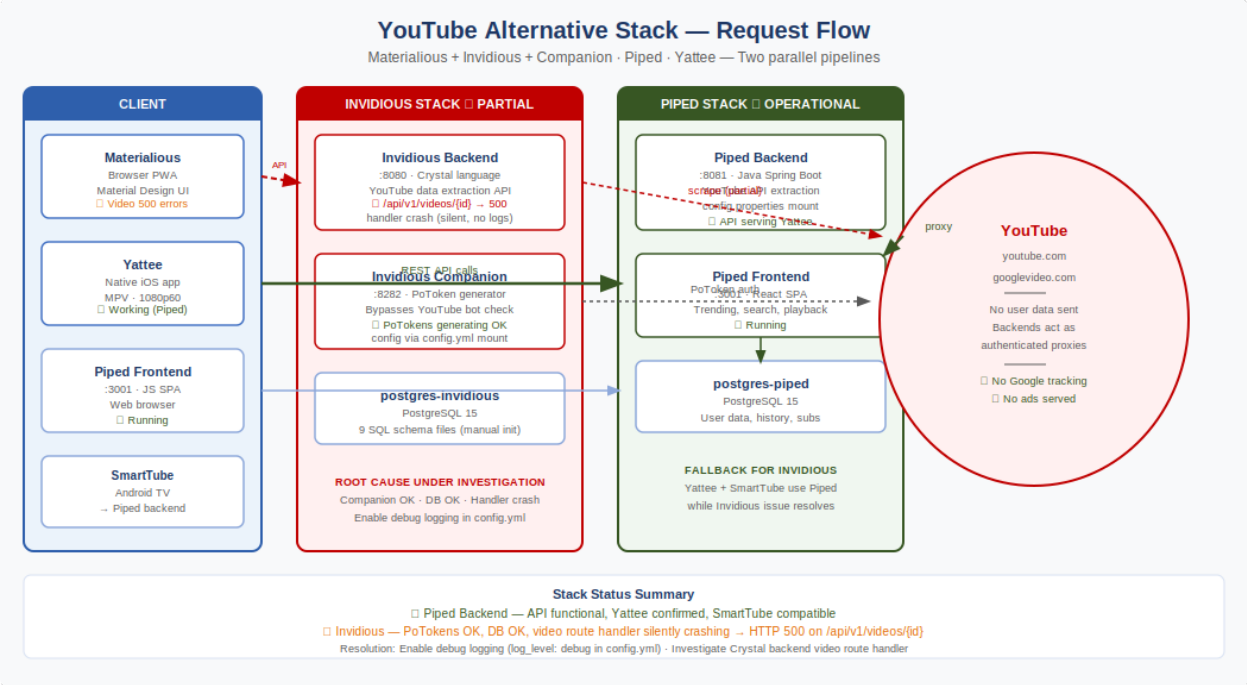

Figure 6: YouTube Stack Flow

6.1 Overview

The YouTube Alternative Stack represents the most technically complex and iteratively developed component of the Jellstrip infrastructure. The goal is to provide a self-hosted, ad-free YouTube frontend accessible on Mac, iPhone, and Android TV, using open-source backends to extract and serve YouTube content without exposing user data to Google. The stack evolved through multiple architecture pivots over several weeks of active development.

6.2 Architecture Evolution

6.2.1 Phase 1 — Piped Standalone

The initial deployment used the TeamPiped Piped-Docker repository. Piped is a fully self-contained YouTube alternative comprising a backend API server (Java Spring Boot), a frontend JavaScript SPA, a PostgreSQL database, and a proxy service for bypassing YouTube's signature checks. The deployment used the configure-instance.sh script provided by the Piped team.

Issue 1 — configure-instance.sh Key Case Bug: The configuration script generated config.properties with keys in SCREAMING_SNAKE_CASE (e.g., API_URL:, PROXY_PART:) instead of the lowercase dot-notation format the Java backend expects. This caused the backend to ignore all configuration values and use defaults, resulting in broken domain routing.

Resolution: config.properties was manually rewritten with correct lowercase keys (apiUrl, proxyPart, etc.).

Issue 2 — YAML Indentation Errors: Multiple docker-compose.yml YAML indentation errors (mapping values are not allowed in this context) caused compose to reject the file entirely. Each error required identifying the specific misaligned line in nano and correcting spacing.

Issue 3 — Domain Routing: The Piped stack requires consistent domain names across the backend, frontend, and proxy configuration. Switching from IP-based access to the youtube.nas local domain required coordinated updates across docker-compose.yml, config.properties, and piped.env.

Partial Success: Trending grid with full thumbnails was achieved and confirmed operational. Video playback remained unreliable.

6.2.2 Phase 2 — Materialious + Invidious

The architecture was pivoted to use Materialious (a Material Design YouTube PWA by wardpearce) as the frontend, backed by an Invidious instance (a Crystal-language YouTube alternative backend) and an Invidious Companion container for PoToken generation.

6.2.3 Current Architecture

Service	Port	Role
postgres-invidious	—	Invidious database (PostgreSQL 15)
invidious-companion	8282	PoToken generation for YouTube authentication bypass
invidious-backend	8080	YouTube data extraction and API server
materialious	3000	Material Design PWA frontend
postgres-piped	—	Piped database (PostgreSQL 15)
piped-backend	8081	Alternative YouTube backend for SmartTube/Android TV
piped-frontend	3001	Piped web frontend

6.3 Key Technical Issues and Resolutions

6.3.1 Invidious Database Initialisation

Invidious's Crystal application does not auto-create its database tables on first run (unlike most web frameworks). The PostgreSQL container initialises with a blank database, and Invidious fails to start with table-not-found errors.

Resolution: All 9 official SQL schema files were manually fetched from the Invidious GitHub repository and concatenated into a single init-sql/01_schema.sql file. This file was mounted into the postgres-invidious container as a docker-entrypoint-initdb.d script, causing PostgreSQL to execute it on first initialisation.

6.3.2 Invidious Companion Configuration

The invidious-companion container requires its configuration (companion URL and secret key) to be passed via a mounted config.yml file. The INVIDIOUS_COMPANION_URL and

INVIDIOUS_COMPANION_KEY Docker environment variables used in earlier deployments do not work for this version of Invidious — the Crystal application reads configuration exclusively from the mounted config.yml file.

Resolution: A config.yml file was created at /opt/homestack/invidious-config/config.yml and mounted into the invidious-backend container. The /companion path suffix is required in the private_url field.

6.3.3 Materialious JSON Parse Crash

Materialious crashed on load with SyntaxError: Unexpected end of JSON input at JSON.parse. The application attempts to read a settings object from localStorage on startup. On a fresh installation, localStorage contains no settings key, causing JSON.parse to receive an empty or null value.

Resolution: Valid JSON was manually written to the localStorage settings key via the browser console, providing the default settings object Materialious expects.

6.3.4 Video Playback 500 Errors

Video route requests to the Invidious API (GET /api/v1/videos/{id}?region=US) consistently return HTTP 500 Internal Server Error. The Invidious Companion container is generating PoTokens successfully (confirmed via logs), but the video route handler in the Invidious Crystal backend is silently crashing before writing its response. The error does not appear in container logs at default log levels.

Current Status: Under investigation. The Piped backend (port 8081) serves as a functional fallback for video retrieval.

6.4 Piped Backend for Mobile Clients

The Piped backend (port 8081) is used as the video backend for the Yattee iOS app. This provides a functional YouTube video pipeline for iPhone use even while the Invidious backend video issue remains unresolved.

6.4.1 Piped Config.properties Issue

The 1337kavin/piped Docker image is a Java Spring Boot application. Spring Boot reads its database and application configuration from a config.properties file mounted into the container. Docker environment variables are not read by the Spring Boot configuration layer — all configuration must be provided via the mounted properties file.

Resolution: A config.properties file was created at /opt/homestack/piped-config/config.properties and mounted into the piped-backend container at /app/config.properties.

6.5 Yattee iOS Client

Yattee is a native iOS YouTube client deployed on the primary user's iPhone, connected to the self-hosted Piped backend. Configuration:

Setting	Value
Primary Engine	MPV — hardware-accelerated, 1080p60 on WiFi and Cellular

AVPlayer Fallback	Enabled for live videos and non-streamable MP4/AVC1 formats
SponsorBlock	Enabled — automatic sponsor/ad segment skipping
Backend	Self-hosted Piped at 192.168.1.90:8081 / 100.88.17.70:8081
cache-secs	120
cache-pause-wait	3
hwidec	auto-safe (hardware decoding)
PWA	Added to iPhone Home Screen as standalone app

Section 7 — Printer Sharing (HP Ink Tank 310)

7.1 Overview

The HP Ink Tank 310 Series printer is shared across the local network via CUPS (Common Unix Printing System) running on the Proxmox host. A socat network socket bridge forwards print data from the network to the printer's USB device node.

7.2 Architecture

- Printer connected via USB to the Proxmox host
- USB device exposed at /dev/usb/lp1
- socat bridge: TCP port 9100 → /dev/usb/lp1
- CUPS configured with HP Ink Tank 310 driver pointing to the socat TCP endpoint
- CUPS web interface available at http://192.168.1.200:631

7.3 Issues and Resolutions

7.3.1 Stuck Print Job — Broken Pipe Error

A print job entered an error state with the message 'Unable to write print data: Broken pipe'. The job appeared in the CUPS queue as a held job and could not be cancelled through the CUPS web interface.

```
cancel -a                                    # Clear all pending jobs
cupsenable HP_Ink_Tank_310_series            # Re-enable the printer queue
lpmove HP_Ink_Tank_310_series-38 HP_Ink_Tank_310_series  # Release held job
```

7.3.2 CUPS Web Interface — Unauthorized

Attempting to manage jobs via the CUPS web interface returned HTTP 401 Unauthorized. This is because CUPS requires administrator-level authentication for job management operations, which the browser session did not have.

Resolution: Job management was performed via command-line CUPS utilities (cancel, cupsenable, lpmove) which execute with root-level privileges on the Proxmox host, bypassing the web interface authentication requirement.

Section 8 — Version Roadmap and Pending Items

8.1 Currently Operational (v1.0)

Component	Status	Notes
Proxmox VE Hypervisor	✓ Operational	Headless, remote management only
AdGuard Home	✓ Operational	Daily use, whole-house DNS filtering
Unbound DNS	✓ ~80%	Running, full integration validation pending
Tailscale Mesh VPN	✓ Operational	Daily use, AdGuard routed through tailnet
Navidrome (Personal)	✓ Operational	~100 FLACs, Amperfy + Feishin clients
Navidrome (Grandpa)	✓ Operational	~1000 songs, family use
Nicotine+ Soulseek	✓ Operational	Active downloads, noVNC GUI
Beets Music Tagger	✓ Operational	Auto-tagging pipeline active
Samba Private Cloud	✓ Operational	Daily use on iPhone and MacBook
Piped Backend	✓ Operational	API serving Yattee iOS client
Jellyfin	✓ Deployed	Running, media library population in progress
Immich	✓ Deployed	Running, photo library population in progress
CUPS Printer Sharing	✓ Operational	HP Ink Tank 310, network shared

8.2 Pending Resolution

Item	Priority	Action Required
Invidious Video 500 Errors	High	Investigate Crystal backend video route handler crash. Enable debug logging in config.yml.
slskd-bot Telegram Conflict	High	Run docker compose up -d --remove-orphans to eliminate orphaned Python bot container.
Jellyfin Media Library	Medium	Populate /mnt/data/movies and /mnt/data/shows with media content.
Immich Photo Population	Medium	Configure iOS Immich app for automatic photo backup over Tailscale.
Gigabit Switch	Medium	Procure 5-port Gigabit switch to eliminate 100Mbps Fast Ethernet bottleneck.

8.3 Future Roadmap

Feature	Priority	Notes
OPNsense VM	High	Full router control, VLANs, firewall rules — bypasses Nokia admin lock

Nginx Proxy Manager / Caddy	High	Local .lan domain names, SSL certificates via DNS-01 challenge
SMB Game Shares	Medium	PS2/PSP ISO streaming via Open PS2 Loader over SMB
Second HDD (RAID/Backup)	Medium	Storage redundancy for media and personal data
Nextcloud / Seafile (Retry)	Low	Revisit after OPNsense and Nginx Proxy Manager are deployed

Section 9 — Configuration Reference

9.1 Complete docker-compose.yml Structure

The complete homestack Docker Compose file is organised into five named stacks within a single /opt/homestack/docker-compose.yml file. This single-file architecture simplifies management — all services are started, stopped, and updated with a single docker compose command, and all relative volume paths share the same /opt/homestack root.

Stack A — Invidious YouTube Backend

```
postgres-invidious  | invidious-companion  | invidious-backend  | materialious
```

Stack B — Piped YouTube Backend

```
postgres-piped  |  piped-backend  |  piped-frontend
```

Stack C — Entertainment

```
jellyfin  |  navidrome  |  navidrome-grandpa  |  nicotine-plus (music-downloader)
```

Stack D — Private Cloud

```
samba
```

Stack E — Photos (Immich Ecosystem)

```
immich-db  |  immich-machine-learning  |  immich-server
```

Supporting Services

```
tailscale-audio  |  unbound-dns  |  beets  |  slskd  |  slskd-bot
```

9.2 Directory Structure — /mnt/data (1.5TB HDD)

```
/mnt/data/
├── music/
│   ├── Sorted/          ← Personal FLAC collection (~100 files)
│   ├── GrandpaMusic/    ← Grandfather's collection (~1000 songs)
│   └── downloads/       ← Nicotine+ download target (Beets input)
├── movies/              ← Jellyfin movie library
├── shows/               ← Jellyfin TV shows library
├── config/
│   ├── jellyfin/
│   ├── navidrome/
│   ├── navidrome-grandpa/
│   ├── nicotine/
│   ├── slskd/
│   └── slskd-bot/
└── samba/               ← Samba config directory
```

9.3 Port Reference

Port	Service	Access URL

8006	Proxmox VE	https://192.168.1.200:8006
8080	AdGuard Home	http://192.168.1.50:8080
9443	Portainer	https://192.168.1.90:9443
5335	Unbound DNS	UDP/TCP 192.168.1.90:5335
8096	Jellyfin	http://192.168.1.90:8096
4533	Navidrome (Personal)	http://192.168.1.90:4533
4534	Navidrome (Grandpa)	http://192.168.1.90:4534
6080	Nicotine+ noVNC	http://192.168.1.90:6080/vnc.html
5030	slskd Web UI	http://192.168.1.90:5030
445	Samba SMB	smb://192.168.1.90/HomeCloud
8083	Immich	http://192.168.1.90:8083
8080	Invidious Backend	http://192.168.1.90:8080
3000	Materialious	http://192.168.1.90:3000
8081	Piped Backend	http://192.168.1.90:8081
3001	Piped Frontend	http://192.168.1.90:3001
631	CUPS Printer	http://192.168.1.200:631

Section 10 — Validation Status Summary

Subsystem	Status	Notes
Proxmox VE Installation	PASS	nomodeset fix applied, stable post-incident
NVRAM/BIOS Recovery	PASS	JBAT1 CMOS reset confirmed effective
Subnet Correction	PASS	192.168.1.200/24 confirmed stable
System Updates (177 packages)	PASS	All packages upgraded via dist-upgrade
AdGuard Home DNS	PASS	Query log active, DoH confirmed, daily use
Unbound DNS	PARTIAL	Container running, full chain validation pending
Tailscale VPN	PASS	100.88.17.70 active, subnet routing approved
Navidrome (Personal)	PASS	100 FLACs streaming, Amperfy + Feishin verified
Navidrome (Grandpa)	PASS	1000 songs serving, independent instance confirmed
Nicotine+ Soulseek	PASS	Online, downloads active, noVNC accessible
Beets Auto-Tagger	PASS	Pipeline active, slskd integration configured
Samba SMB Cloud	PASS	iPhone and MacBook access confirmed daily
Jellyfin	DEPLOYED	Running, media library pending
Immich	DEPLOYED	Running, photo library pending
slskd Engine	PASS	Connected as MyHomeLabUser2026
slskd-bot Telegram	PENDING	Orphan bot conflict requires cleanup
Piped Backend	PASS	API functional, Yattee client confirmed
Materialious Frontend	PARTIAL	UI functional, video playback 500 errors
Invidious Backend	PARTIAL	Running, video route handler crashing silently
CUPS Printer Sharing	PASS	HP Ink Tank 310 shared, stuck jobs resolved
Seafile (Abandoned)	FAIL	Circular symlink bug, replaced by Samba